Type System vs. Type Declaration

The type declaration system in PHP 8 is founded upon the built-in data types of the language, with extra features to make life easier (and code safer). PHP 8 offers us a range of base types: integers, floats, strings, null, booleans, arrays, and objects (together with the `callable` and `resource` special types).

Declarations will also accept the name of a `class`, `interface`, or `enum` (known as *class types* or *user-defined types*, although the latter is something of a misnomer because it includes built-in classes and interfaces). While it's true that a type declaration of `object` would suffice, often it's the class of the object that is of more interest. Allowing any old object into your function might be a recipe for disaster if your code is expecting the object to behave in specific ways. For example, a function might be coded to work with an instance of `DateTime`, which is passed in at call time. By declaring the argument type as `DateTime`, you can prevent an incompatible object from being passed in and generating errors.

PHP also offers a range of *standalone* pseudo-types, which combine with the primitives to form a set of *base types*. "Standalone" simply means "not part of a composite" (i.e., not part of a set of types; I promise this will make sense very shortly). A couple of the standalone pseudo-types are used exclusively for declaring how a function behaves when it completes execution, and these are known as *return-only types*.

Composite types are also supported. Composite types are pseudo-types too; they exist only to serve the needs of functions (and objects), and they define sets of types in declarations. A function might, say, return a string or null, or might expect an incoming object to implement multiple interfaces. Composite types are expressions that define sets of types in various logical combinations.

This is a lot to take in, but hopefully it will be clearer with some examples. Let's start with base types.

Base Types

Before we look at the mechanisms of declaration in detail, let's quickly look at the base types of the declaration system, starting with the built-in data types.

- `null` (only allowed as standalone with PHP >= 8.2)
- `bool`